FREELAJOB

Sistema de Marketplace Freelancer

Documentação Técnica Completa

Modelagem de Banco de Dados | Arquitetura | Código | API | Tutoriais

Versão: 1.0

Data: Novembro de 2026

Stack: Laravel 11 + PHP 8.4 + MySQL 8.0 + Tailwind CSS + Alpine.js

Pagamento: PagSeguro (Pix, Cartão, Boleto) - Custódia/Escrow

Banco de Dados: MySQL (C:\laragon\www\freelajob)

Este documento contém a documentação técnica completa do sistema FreelaJob, incluindo modelagem de banco, arquitetura, padrões de design, documentação da API e tutoriais de uso.

Sumário

1. 1. Visão Geral do Sistema
2. 2. Modelagem do Banco de Dados
3. 3. Arquitetura e Padrões de Design
4. 4. Guia de Instalação
5. 5. Documentação da API
6. 6. Tutoriais de Uso
7. 7. Segurança e Permissões
8. 8. Logs e Auditoria
9. 9. Roadmap e Próximos Passos

Fontes: Código-fonte em C:\laragon\www\freelajob | Baseado em análise do 99Freelas

1. Visão Geral do Sistema

O **FreelaJob** é uma plataforma marketplace que conecta clientes a freelancers com pagamento seguro em custódia (escrow), similar ao 99Freelas.

O fluxo principal é:

- Cliente publica um projeto com descrição detalhada
- Plataforma aprova o projeto para receber propostas
- Freelancers enviam propostas com valor e prazo
- Cliente seleciona o melhor freelancer
- Cliente paga o valor, que fica retido em custódia
- Freelancer executa o projeto e marca como concluído
- Cliente libera o pagamento após verificar a entrega
- Plataforma repassa o valor ao freelancer (com taxa de intermediação)

Atores do sistema:

Ator	Responsabilidades
Cliente	Publica projetos, contrata freelancers, paga em custódia, libera pagamento, avalia.
Freelancer	Envia propostas, executa projetos, entrega trabalhos, recebe pagamentos, monta portfólio.
Admin	Modera projetos, gerencia usuários, pagamentos, disputas, planos, visualiza gráficos.

2. Modelagem do Banco de Dados

O banco de dados MySQL contém 18 tabelas principais:

Tabela	Descrição
users	Usuários (cliente, freelancer, admin) - auth, tipo, status, api_token
user_profiles	Perfis dos usuários - foto, bio, telefone, cidade, Pix, conta bancária, CV
categories	Categorias de projetos - nome, slug, ícone, cor, descrição
skills	Habilidades - nome, slug, categoria_id
projects	Projetos - título, descrição, status, orçamento, prazo, cliente, categoria
project_skill	Pivot projeto-habilidade (muitos-para-muitos)
project_attachments	Anexos dos projetos - arquivo, nome, tamanho, tipo MIME
proposals	Propostas - valor, prazo, detalhes, status, freelancer, projeto
payments	Pagamentos em custódia - valor, taxas, método, status, transaction_id
messages	Mensagens do chat por projeto
plans	Planos de assinatura - preço, comissão, limites, dias para saque
plan_features	Funcionalidades de cada plano
user_plan_subscriptions	Assinaturas ativas dos usuários
disputes	Disputas entre cliente e freelancer
reviews	Avaliações 1-5 estrelas entre usuário e freelancer
withdrawals	Saque de saldos dos freelancers via Pix
portfolio_items	Itens de portfólio dos freelancers
notifications	Notificações do sistema
audit_logs	Log de auditoria de todas as ações

Relacionamentos principais:

- users 1:N user_profiles, projects, proposals, payments, messages, withdrawals, portfolio_items, notifications, audit_logs, subscriptions
- categories 1:N projects, skills
- skills 1:N project_skill
- projects 1:N proposals, payments, messages, project_attachments, reviews, disputes

- proposals 1:N payments
- payments 1:N withdrawals
- plans 1:N plan_features, user_plan_subscriptions
- users 1:N user_plan_subscriptions, reviews, disputes

3. Arquitetura e Padrões de Design

O sistema segue os princípios SOLID e utiliza os seguintes padrões de design:

Strategy: Pagamentos: Pix, Cartão, Boleto são estratégias intercambiáveis (PaymentStrategyInterface).

State: Projetos: máquina de estados (aguardando_aprovacao → aguardando_propostas → aguardando_pagamento → em_andamento → aguardando_liberação → concluído).

Observer: Notificações: eventos de pagamento, proposta, disputa disparam observadores (e-mail, in-app).

Builder: Criação de projetos e propostas com muitos campos opcionais.

Facade: PaymentService simplifica o uso das estratégias de pagamento.

Decorator: Auditoria: envolve serviços para logar ações automaticamente.

Camadas do sistema:

Camada	Responsabilidade	Exemplos
Routes	Definição de endpoints web e API	routes/web.php, routes/api.php
Controllers	Recebe requisições, chama serviços, retorna views/JSON	AuthController, ProjectController, PaymentController
Services	Lógica de negócio complexa (pagamentos, custódia)	PaymentService
Models	Entidades Eloquent e relacionamentos	User, Project, Payment, Proposal
Requests	Validação de entrada de dados	RegisterRequest, ProjectRequest, PlanRequest
Resources	Transformação de modelos em JSON para API	ProjectResource, UserResource, PaymentResource
Middleware	Autenticação, status de usuário, auditoria	api.token, status, audit.logger
Views (Blade)	Templates HTML com Tailwind + Alpine.js	dashboard, projetos, pagamentos

4. Guia de Instalação

Pré-requisitos:

- PHP 8.2+
- Composer
- MySQL 8.0
- Node.js + npm
- Laragon (recomendado para Windows)

Passos para instalar:

```
# 1. Clonar o repositório git clone https://github.com/michelcodingDev/freela.git cd freela # 2. Instalar dependências PHP composer install # 3. Instalar dependências JS npm install # 4. Copiar arquivo de ambiente cp .env.example .env # 5. Gerar chave da aplicação php artisan key:generate # 6. Configurar banco de dados no .env DB_CONNECTION=mysql DB_HOST=127.0.0.1 DB_PORT=3306 DB_DATABASE=freelajob DB_USERNAME=root DB_PASSWORD= # 7. Rodar migrações e seeders php artisan migrate --seed # 8. Build dos assets frontend npm run build # 9. Iniciar o servidor php artisan serve
```

Usuário admin padrão:

E-mail: admin@freelajob.com

Senha: admin123

5. Documentação da API

A API REST está disponível em **/api** e utiliza autenticação via **api_token** (header Bearer).

Endpoints disponíveis:

Método	Endpoint	Descrição
POST	/api/auth/register	Cadastro de usuário (cliente/freelancer)
POST	/api/auth/login	Login e obtenção de token
POST	/api/auth/logout	Logout (invalida token)
GET	/api/me	Dados do usuário autenticado
GET	/api/profile	Perfil do usuário
PUT	/api/profile	Atualizar perfil
GET	/api/projetos	Listar projetos (filtros: categoria, busca)
GET	/api/projetos/{id}	Detalhes de um projeto
GET	/api/projetos/categorias	Listar categorias
GET	/api/projetos/habilidades	Listar habilidades
GET	/api/projetos/{id}/propostas	Listar propostas (apenas cliente)
POST	/api/projetos/{id}/propostas	Enviar proposta (apenas freelancer)
POST	/api/projetos/{id}/pagamentos	Criar pagamento em custódia
GET	/api/pagamentos	Listar pagamentos do usuário
POST	/api/pagamentos/{id}/liberar	Liberar pagamento (apenas cliente)

Exemplo de requisição (login):

```
curl -X POST http://localhost:8000/api/auth/login \ -H "Content-Type: application/json" \ -d '{"email":"cliente@email.com","password":"senha123"}' # Resposta: # { # "success": true, # "user": { "id": 1, "name": "João", "type": "cliente", ... }, # "token": "80_caracteres_aleatorios" # }
```

Exemplo de requisição autenticada (listar projetos):

```
curl -X GET http://localhost:8000/api/projetos \ -H "Authorization: Bearer SEU_API_TOKEN"
```

6. Tutoriais de Uso

6.1 Para Clientes

- Cadastre-se como Cliente
- Publique um projeto (título, descrição, categoria, orçamento, prazo)
- Aguarde aprovação da plataforma
- Receba e avalie propostas dos freelancers
- Selecione o melhor freelancer
- Pague o valor (Pix, Cartão ou Boleto) - fica em custódia
- Acompanhe a execução via chat
- Liberar pagamento apenas após verificar a entrega
- Avalie o freelancer (1-5 estrelas)

6.2 Para Freelancers

- Cadastre-se como Freelancer
- Complete o perfil (título profissional, bio, portfólio, chave Pix)
- Navegue pelos projetos disponíveis
- Envie propostas com valor e prazo
- Aguarde seleção do cliente
- Execute o projeto após confirmação de pagamento
- Marque como concluído e entregue os arquivos

- Receba o pagamento após liberação do cliente
- Saque o saldo via Pix

6.3 Para Admin

- Acesse o dashboard admin para visão geral
- Aprove/reprova projetos publicados
- Gerencie usuários (ativar, suspender, banir)
- Monitore todos os pagamentos em custódia
- Crie e edite planos de assinatura
- Resolva disputas entre cliente e freelancer
- Exporte logs de auditoria em CSV
- Acompanhe gráficos de receita e crescimento

7. Segurança e Permissões

Permissões por tipo de usuário:

Permissão	Cliente	Freelancer	Admin
Ver usuários	Sim	Sim	Sim
Criar projetos	Sim	Não	Sim
Enviar propostas	Não	Sim	Não
Fazer pagamentos	Sim	Não	Sim
Liberar pagamentos	Sim	Não	Sim
Ver todos pagamentos	Não	Próprios	Sim
Gerenciar planos	Não	Não	Sim
Resolver disputas	Não	Não	Sim

Medidas de segurança implementadas:

- Pagamento em custódia (escrow) - valor retido até entrega
- Bloqueio de troca de contato externo no chat
- API com token de autenticação (api_token)
- Audit log de todas as ações de mutação
- Validação de entrada com Form Requests
- Sanitização de dados sensíveis (senhas removidas dos logs)
- Middleware de status de usuário (banido/suspenso)
- CSRF protection em todas as rotas web

8. Logs e Auditoria

O sistema mantém dois tipos de logs:

8.1 Auditoria de Ações (tabela audit_logs)

Todas as ações de mutação (POST, PUT, PATCH, DELETE) são registradas automaticamente pelo AuditLoggerMiddleware, incluindo: usuário, ação, modelo, ID, valores antigo/novo, IP e user agent.

8.2 Logs de Erro do Sistema

Erros e exceções são gravados em storage/logs/laravel.log e podem ser visualizados no painel admin em Logs de Erro.

8.3 Exportação

Os logs de auditoria podem ser exportados em CSV via painel admin.

9. Roadmap e Próximos Passos

MVP (Concluído): Cadastro, projetos, propostas, pagamentos Pix manual, dashboard admin com gráficos, API para app mobile

Fase 2: Gateway automático (Mercado Pago/Efi/Stripe), disputas completas, planos pagos, turbinas, notificações push

Fase 3: App mobile (Play Store), ranking público, IA para recomendação, fluxo de caixa previsto

Documentação gerada em novembro/2026. Código-fonte em C:\laragon\www\freelajob.